🚀 Week 1: Core Python Foundations

Day 1 Challenge: Word Frequency Counter

Overview The foundation of data engineering is text processing. You will build a tool that reads a raw text file and identifies which words appear most often, which is the first step toward building search engines or sentiment analyzers.
Challenge Requirements * Create `data.txt` with a paragraph of text.

- Write `word_counter.py` to:
  - Load the file.
  - Normalize text (lowercase and remove punctuation).
  - Count occurrences using a dictionary.
- **The Twist (Sub-Challenge): The Stop-Word Filter.** Create a list called `stop_words = ["the", "is", "at", "which", "on"]`. Modify your script so it **excludes** these common words from your final count.

Hints * Use `.split()` to turn a string into a list of words.

- Use `my_dict.get(word, 0) + 1` to update counts safely.

Goal Master basic File I/O, dictionary manipulation, and data cleaning.

Day 2 Challenge: Custom CSV Parser

Overview Before using heavy libraries like Pandas, you must understand the "anatomy" of a CSV. This challenge tasks you with breaking down a comma-separated string into structured parts manually.
Challenge Requirements * Create `players.csv` with columns: `Name, Team, Runs`.

- Write `csv_parser.py` to:
  - Read the file line by line.
  - Split each line by the comma delimiter.
  - Store each row as a list within a master list.
- **The Twist (Sub-Challenge): The Header Mapper.** Instead of a list of lists, modify the parser to return a **list of dictionaries**, where the keys are the header names (e.g., `{"Name": "Virat", "Team": "RCB", "Runs": "100"}`).

Hints * Use `next(file_object)` to skip or capture the first header line.

- `zip()` is a very helpful function for mapping headers to row values.

Goal Understand row-based data structures and manual data parsing.

Day 3 Challenge: List Comprehensions on IPL Stats

Overview Data engineers often need to transform large lists quickly. List comprehensions are the "Pythonic" way to filter and modify data without writing long, slow `for` loops.
Challenge Requirements * Use your list of dictionaries from Day 2.

- Write `comprehensions.py` to:

    - Create a new list containing only the names of players who scored > 50 runs.
    - Create a second list where all team names are converted to uppercase.
- **The Twist (Sub-Challenge): The Conditional Scoreboard.** Create a list of strings that says "High Scorer" if runs > 100, and "Low Scorer" if runs <= 100, for every player in the dataset.

Hints * Syntax: [expression for item in iterable if condition].

- Remember that CSV values are read as strings; use int() before comparing runs.

Goal Write concise, readable, and efficient transformation logic.

Day 4 Challenge: Merge Multiple CSVs

Overview In production, data rarely arrives in one file. You will often receive daily "drops." This challenge simulates an ETL (Extract, Transform, Load) task of merging fragmented files.

Challenge Requirements * Create day1.csv and day2.csv with the same headers.

- Write merger.py to:
    - Read both files.
    - Combine the records into one list.
    - Write the result to all_days.csv.
- **The Twist (Sub-Challenge): The Unique Identifier.** If a player appears in both files, only write their **latest** entry (from Day 2) to the final file to avoid duplicate records.

Hints * Open the first file in w (write) mode to set headers, then open the rest in a (append) mode.

- Dictionaries are great for deduplication because keys must be unique.

Goal Handle multiple file streams and implement basic deduplication logic.

Day 5 Challenge: Robust Error Handling (Try/Except)

Overview Production pipelines fail. Files go missing, or rows contain letters where numbers should be. This challenge teaches you how to keep a script running even when the data is "dirty."

Challenge Requirements * Write robust_parser.py that attempts to open a non-existent file.

- Use try...except to catch the FileNotFoundError and print a friendly message.
- **The Twist (Sub-Challenge): The Value Guard.** While parsing your CSV, if a "Runs" column contains text (like "DNA" or "Missing") instead of a number, catch the ValueError, skip that row, and log the "bad row" to a separate file called skipped_rows.txt.

Hints * Wrap only the "risky" line of code in the try block.

- Use finally to ensure the file closes no matter what happens.

Goal Build "defensive" code that handles unexpected inputs without crashing.

Day 6 Challenge: OOP Match Attributes

Overview Object-Oriented Programming (OOP) allows you to model real-world entities. Instead of managing raw strings or lists, you will create a "Match" blueprint.
Challenge Requirements * Define a class `Match`.

- Initialize it with `team_a`, `team_b`, `score_a`, and `score_b`.
- **The Twist (Sub-Challenge): The Score Delta.** Add a method called `get_margin()` that returns the absolute difference between the two scores. Then, create a list of 3 `Match` objects and print the margin for each.

Hints * Use the `__init__` method to set your attributes.

- Use `abs(self.score_a - self.score_b)` for the margin.

Goal Understand how to encapsulate data and logic within Python Classes.

Day 7 Challenge: Mini Project — IPL Stats Analyzer

Overview This is the "Final Exam" for Week 1. You will combine everything—File I/O, CSV Parsing, List Comprehensions, Error Handling, and OOP—into one working application.
Challenge Requirements * **Extract:** Read a CSV file of match results.

- **Transform:** Convert each row into a `Match` object. Use list comprehensions to filter out matches played in a specific year.
- **Load:** Print a summary of the "Total Matches Played" and the "Average Winning Margin."
- **The Twist (Sub-Challenge): The Error Summary.** At the end of the script, report how many "bad rows" were skipped during the process (using the logic from Day 5).

Hints * Break your code into functions: `load_data()`, `process_stats()`, and `main()`.

- This is your first "Pipeline."

Goal Successfully integrate multiple Python concepts into a single, functional data workflow.

Ready to start? You can find a sample IPL dataset to download and use in VS Code here: [IPL Matches Dataset (Kaggle/GitHub)]

To provide you with a structured document ready for PDF export in **VS Code**, I have detailed the remaining weeks (Weeks 2–4) in your requested format.
To turn this into a PDF:

1. Copy the text below into a file named `30DayPythonChallenge.md` in VS Code.
2. Use the **Markdown PDF** extension (or press `Ctrl+Shift+P` and type `Markdown PDF: Export (pdf)`).

🚀 30-Day Python for Data Engineering (Weeks 2-4)

Week 2: Advanced Python Concepts

Day 8 Challenge: Generator for Streaming Log Lines

Overview Data Engineers often handle files too large for RAM. Instead of loading an entire 10GB log file, you will build a **Generator** to "stream" lines one by one.
Challenge Requirements

- Create a large dummy log file `app.log`.
- Write `log_streamer.py` using the `yield` keyword.
- Iterate through the generator to print only lines containing "ERROR".
- **The Twist (Sub-Challenge): The Counter-Streamer.** Modify the generator to yield a tuple: `(line_number, line_content)`. This helps in debugging exactly where an error occurred in a massive file.

Hints

- A generator function uses `yield` instead of `return`.
- Use `enumerate()` to track line numbers easily.

Day 9 Challenge: Decorator for Timing Functions

Overview Performance tuning is a core part of ETL. You will create a **Decorator**—a reusable wrapper—that measures how long any specific function takes to execute.
Challenge Requirements

- Write a function `heavy_computation()` that simulates work using `time.sleep()`.
- Create a decorator `@timer` that prints the execution time in seconds.
- **The Twist (Sub-Challenge): The Threshold Alert.** Modify the decorator to accept an argument. If the function takes longer than 2.0 seconds, print a warning: `[PERFORMANCE ALERT] Function exceeded limit!`.

Hints

- Use `time.perf_counter()` for high-precision timing.
- Remember to use `functools.wraps` to preserve function metadata.

Day 10 Challenge: Context Manager for File Handling

Overview Leaking file handles or database connections causes system crashes. You will learn to use the `with` statement by building a custom **Context Manager**.
Challenge Requirements

- Create a class `FileManager` with `__enter__` and `__exit__` methods.
- Ensure the file is automatically closed even if an error occurs inside the `with` block.
- **The Twist (Sub-Challenge): The Mode Guard.** Modify `__enter__` so that it checks if the file exists before opening. If it doesn't, it should create the file automatically and log: `"New file initialized"`.

Goal Understand resource lifecycle management in production scripts.

Day 11 Challenge: Dataclasses for Player Info

Overview Standard classes require a lot of boilerplate code (like `__init__` and `__repr__`). **Dataclasses** simplify this, making your data structures cleaner and more readable.
Challenge Requirements

- Define a `@dataclass` for `PlayerStats` including `name`, `runs`, and `matches`.
- **The Twist (Sub-Challenge): Automatic Average.** Use the `__post_init__` method to calculate a `batting_average` field automatically as soon as the object is created.

Hints

- Import `dataclass` from the `dataclasses` module.
- Handle the case where `matches` might be zero to avoid `ZeroDivisionError`.

Day 12 Challenge: Type Hints in Functions

Overview Data engineering projects are often large. **Type Hints** act as documentation and help tools like VS Code catch bugs before you even run your code.

Challenge Requirements

- Take your CSV parser from Day 2 and add type annotations to all parameters and return values.
- **The Twist (Sub-Challenge): Strict Validation.** Use the `typing` module (e.g., `List`, `Dict`, `Optional`) to ensure a function can return either a dictionary of stats OR `None` if the player isn't found.

Goal Transition from "scripting" to "software engineering" standards.

Week 3: Production Practices

Day 15 Challenge: Replace print() with Logging

Overview In a professional environment, `print()` is ignored. **Logging** allows you to categorize messages by importance (Info, Warning, Error) and save them to persistent files.

Challenge Requirements

- Setup `logging.basicConfig` to save logs to `pipeline.log`.
- **The Twist (Sub-Challenge): Multi-Destination Logging.** Configure the logger so that "INFO" messages go to the console, but "ERROR" messages are saved to both the console AND a dedicated `critical_errors.log` file.

Hints

- Explore `logging.FileHandler` and `logging.StreamHandler`.

Day 16 Challenge: Unit Tests with Pytest

Overview If your code isn't tested, it’s broken. You will use **Pytest** to verify that your data transformation logic produces the correct results every time.

Challenge Requirements

- Write a function `clean_currency(value)` that removes `$` and `,` from strings and returns a float.
- Create a `test_cleaner.py` file with 3-4 test cases using `assert`.
- **The Twist (Sub-Challenge): The Exception Test.** Use `with pytest.raises(ValueError):` to verify that your function correctly handles (and fails on) completely invalid input like the string `"ABC"`.

Goal Ensure your data pipeline doesn't produce "Silent Failures."

Day 18 Challenge: Command-Line Options with Argparse

Overview Instead of hardcoding filenames inside your script, you will use **Argparse** to pass parameters directly from the terminal.

Challenge Requirements

- Modify your CSV parser to accept a path via the command line: `python parser.py --input data.csv`.
- **The Twist (Sub-Challenge): The Format Switch.** Add an argument `--format` that allows the user to choose between `csv` and `json` output. If the user picks an unsupported format, show a helpful error message.

Hints

- Use `parser.add_argument()` with the `choices` parameter.

Week 4: Data Engineering Python

Day 22 Challenge: Connect to SQLite

Overview Data Engineers move data from files to databases. You will use the built-in `sqlite3` module to create a local database and store your processed CSV data.

Challenge Requirements

- Connect to `stats.db`.
- Create a table `players` and insert data using `execute()`.
- **The Twist (Sub-Challenge): Batch Loading.** Instead of inserting rows one by one in a loop, use `executemany()` to load a list of 100 rows instantly. This is significantly faster for large datasets.

Goal Transition from flat-file storage to relational database management.

Day 24 Challenge: Fetch Data from an API

Overview Modern data pipelines often pull data from web services. You will use the `requests` library to fetch JSON data from a live API.

Challenge Requirements

- Fetch data from: `https://jsonplaceholder.typicode.com/users`.
- Extract the names and email addresses.
- **The Twist (Sub-Challenge): The Status Checker.** Wrap your request in a function that checks the `status_code`. If the API is down (500) or the page is missing (404), raise a custom exception rather than letting the script crash.

Dataset Link: [JSONPlaceholder API]

Day 29 Challenge: Mini Project — End-to-End ETL Pipeline

Overview This is the final test. You will build a complete **Extract, Transform, Load** pipeline that mirrors a real-world data engineering task.

Challenge Requirements

1. **Extract:** Fetch user data from an API.
2. **Transform:** Use a list comprehension to filter users who live in a specific "City." Add a timestamp of when the data was processed.
3. **Load:** Save the cleaned, filtered data into a SQLite database.

- **The Twist (Sub-Challenge): The Integrity Check.** Before loading into the database, write a "Validator" function that ensures no email address is a duplicate. If a duplicate is found, log it as a "Warning" and skip it.

Goal Prove you can handle the entire lifecycle of data using Python.

🔗 Recommended Datasets for your VS Code project:

- **IPL Cricket Data:** Download CSV
- **Web Logs (Apache):** Download Log Sample
- **JSON API for Testing:** https://jsonplaceholder.typicode.com/posts

Follow-up Question:
Which of these specific days (e.g., the API fetch or the SQL connection) would you like me to provide a code template for to get you started in VS Code?